

Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Сибирский государственный университет телекоммуникаций и
информатики»
(СибГУТИ)

Кафедра ТСиВС

ОТЧЕТ ПО ИТОГОВОМУ ПРОЕКТУ

**По дисциплине: «Интернет-технологии и сетевое
программное обеспечение»**

Тема: «Сервис логирования»

Студент группы ИКС-433

ФИО студента Демин С.А.

Проверил:

Должность Старший преподаватель

ФИО Лошкарев А. В.

Новосибирск 2026 г.

Оглавление

Введение	3
1 Выполнение работы	4
1.1 Архитектура системы	4
1.2 Контейнеризация и запуск	5
1.3 Проверка	6
1.3.1 Результаты тестирования	6
1.3.2 Запуск контейнеров	7
1.3.3 Создание логов и работа notifier	7
1.3.4 Просмотр списка логов, сводки и уведомлений	7
Заключение	9

Введение

Цель работы

Целью работы является разработка и сборка микросервисного проекта для работы с логами, в котором внешний REST API, внутреннее gRPC-взаимодействие, контейнеризация и запуск через docker-compose объединены в одну целостную систему.

В рамках работы была реализована микросервисная система для работы с логами. Проект состоит из трёх компонентов: gateway, logs-svc-s05 и log-notifier.

Сервис gateway выступает как единая точка входа для внешних запросов. Он принимает обращения к API и перенаправляет их в основной сервис.

logs-svc-s05 отвечает за основную логику. Он хранит логи в памяти, умеет добавлять новые записи, отдавать список логов, строить сводку по уровням INFO, WARNING и ERROR, получать логи по уровню, удалять запись по идентификатору и обращаться к log-notifier по gRPC после добавления новой записи.

log-notifier используется как внутренний сервис уведомлений. Он принимает запрос о создании лога и подтверждает получение сообщения. Для внешнего взаимодействия используется REST, а для внутреннего обмена между сервисами gRPC. Проект упакован в Docker и запускается одной командой.

1 Выполнение работы

1.1 Архитектура системы

В проекте реализована микросервисная система для работы с логами. Она состоит из трёх сервисов: gateway, logs-svc-s05 и log-notifier.

Gateway принимает внешние запросы и перенаправляет их в основной сервис. Он нужен для того, чтобы отделить внешний интерфейс от внутренней логики приложения. Logs-svc-s05 хранит логи в памяти, умеет добавлять новые записи, отдавать список и сводку по уровням, получать логи по уровню, удалять запись по идентификатору, а также отправляет уведомление в log-notifier после добавления записи. Если внутренний сервис недоступен, основной сервис продолжает обрабатывать запросы.

Связь между сервисами построена следующим образом: клиент обращается к gateway, тот перенаправляет запросы в logs-svc-s05, а logs-svc-s05 вызывает log-notifier по gRPC. Такой подход позволяет разделить ответственность между компонентами и показать работу двух разных протоколов в одном проекте. Для внешнего API используется REST, а для внутреннего взаимодействия между сервисами - gRPC.

Доступные эндпоинты API

Сервис logs-svc-s05 предоставляет следующие REST-эндпоинты:

- POST /api/logs/ - создание новой записи лога
- GET /api/logs/ - получение списка всех логов
- GET /api/logs/summary - получение сводки с общим количеством логов и разбивкой по уровням
- GET /api/logs/{id} - получение конкретной записи лога по идентификатору

- GET /api/logs/level/{level} - получение логов только для заданного уровня
- DELETE /api/logs/{id} - удаление лога по идентификатору
- GET /api/notifications - получение истории уведомлений, полученной от notifier-сервиса

Все запросы к API проходят через gateway, который выступает единой точкой входа.

Проверка запросов через curl

Для быстрой проверки работы системы подготовлены следующие HTTP-запросы:

```
curl http://localhost:8080/api/logs/  
curl http://localhost:8080/api/logs/summary  
curl http://localhost:8080/api/logs/level/INFO  
curl http://localhost:8080/api/logs/1  
curl -X DELETE http://localhost:8080/api/logs/1  
curl http://localhost:8080/api/notifications  
curl "http://localhost:8080/api/notifications?limit=5"
```

Для создания новой записи используется POST-запрос:

```
curl -X POST http://localhost:8080/api/logs/ -H "Content-Type: application/json" -d "{\"message\":\"test log\",\"level\"
```

1.2 Контейнеризация и запуск

Для локального запуска подготовлен файл docker-compose.yml, который поднимает все сервисы одной командой. Каждый сервис вынесен в отдельный контейнер со своим Dockerfile, поэтому компоненты можно запускать и проверять независимо друг от друга.

Команда для запуска проекта:

```
docker-compose up --build
```

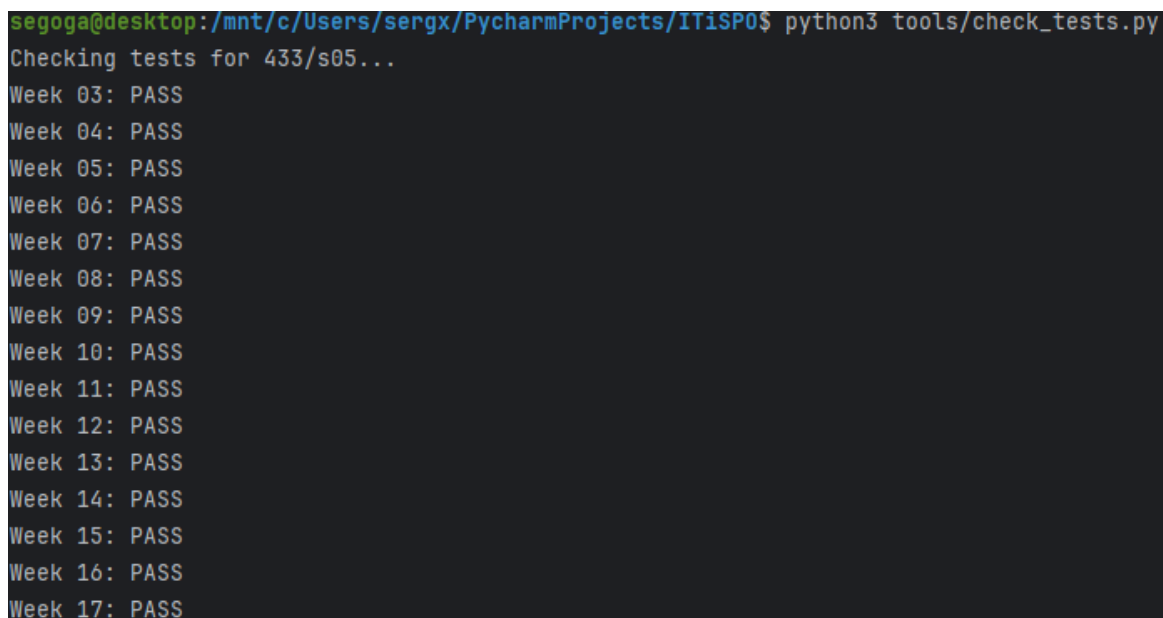
После запуска доступен gateway на порту 8080, через который можно отправлять запросы к сервису логов. Дополнительно подготовлены Kubernetes-манифесты для развертывания системы в кластере.

1.3 Проверка

Тесты практических работ пройдены успешно. Отдельно были проверены: запуск контейнеров через docker-compose, работа REST-эндпоинтов, получение истории уведомлений через `/api/notifications` и передача сообщений между сервисами через gRPC.

1.3.1 Результаты тестирования

При запуске всех тестов практических работ с 1 по 17 неделю все проверки завершились успешно.



```
segoga@desktop:/mnt/c/Users/sergx/PycharmProjects/ITiSP0$ python3 tools/check_tests.py
Checking tests for 433/s05...
Week 03: PASS
Week 04: PASS
Week 05: PASS
Week 06: PASS
Week 07: PASS
Week 08: PASS
Week 09: PASS
Week 10: PASS
Week 11: PASS
Week 12: PASS
Week 13: PASS
Week 14: PASS
Week 15: PASS
Week 16: PASS
Week 17: PASS
```

Рисунок 1.1 – Результаты выполнения тестов

1.3.2 Запуск контейнеров

При выполнении команды `docker-compose up --build` происходит сборка образов для всех трёх сервисов, создание сети и запуск контейнеров.

```
[+] up 7/7
✓ Image week-17-log-notifier      Built
✓ Image week-17-gateway          Built
✓ Image week-17-logs-svc-s05     Built
✓ Network week-17_default        Created
✓ Container week-17-log-notifier-1 Created
✓ Container week-17-logs-svc-s05-1 Created
✓ Container week-17-gateway-1    Created
Attaching to gateway-1, log-notifier-1, logs-svc-s05-1
log-notifier-1 | log-notifier запущен на порту 50051
gateway-1      | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
gateway-1      | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
gateway-1      | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
gateway-1      | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
gateway-1      | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
gateway-1      | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
gateway-1      | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
gateway-1      | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
gateway-1      | /docker-entrypoint.sh: Configuration complete; ready for start up
logs-svc-s05-1 | INFO: Started server process [1]
logs-svc-s05-1 | INFO: Waiting for application startup.
logs-svc-s05-1 | INFO: Application startup complete.
logs-svc-s05-1 | INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Рисунок 1.2 – Запуск сервисов через docker compose

1.3.3 Создание логов и работа notifier

При отправке POST-запросов на создание логов:

- gateway логирует входящие запросы с указанием метода, пути и статуса ответа
- logs-svc-s05 логирует информацию о создании каждой записи
- log-notifier при получении gRPC-уведомления выводит сообщение с id, уровнем и текстом полученного лога

1.3.4 Просмотр списка логов, сводки и уведомлений

После создания нескольких тестовых записей:

- GET /api/logs/ возвращает все созданные логи

```

log-notifier-1 | received log id=1 level=INFO message=test log 1

gateway-1      | 172.24.0.1 - - [28/May/2026:12:35:56 +0000] "POST /api/logs/ HTTP/1.1" 201 46 "-" "curl/8.5.0"
logs-svc-s05-1 | INFO:      172.24.0.4:38532 - "POST /logs/ HTTP/1.0" 201 Created

log-notifier-1 | received log id=2 level=WARNING message=test log 2
gateway-1      | 172.24.0.1 - - [28/May/2026:12:36:08 +0000] "POST /api/logs/ HTTP/1.1" 201 49 "-" "curl/8.5.0"
logs-svc-s05-1 | INFO:      172.24.0.4:50272 - "POST /logs/ HTTP/1.0" 201 Created

gateway-1      | 172.24.0.1 - - [28/May/2026:12:36:18 +0000] "POST /api/logs/ HTTP/1.1" 201 47 "-" "curl/8.5.0"
log-notifier-1 | received log id=3 level=ERROR message=test log 3

```

Рисунок 1.3 – Создание логов и работа notifier

- GET /api/logs/summary возвращает общее количество логов и разбивку по уровням
- GET /api/notifications возвращает историю уведомлений, собранную notifier-сервисом

```

segoga@NoteBook: /mnt/c/Users/sergx/PycharmProjects/ITiSP0/weeks/week-17$ curl http://localhost:8080/api/logs/
[{"message":"test log 1","level":"INFO","id":1}, {"message":"test log 2","level":"WARNING","id":2}, {"message":"test log 3","level":"ERROR","id":3}]
segoga@NoteBook: /mnt/c/Users/sergx/PycharmProjects/ITiSP0/weeks/week-17$
segoga@NoteBook: /mnt/c/Users/sergx/PycharmProjects/ITiSP0/weeks/week-17$ curl http://localhost:8080/api/logs/summary
{"total":3,"by_level":{"INFO":1,"WARNING":1,"ERROR":1}}
segoga@NoteBook: /mnt/c/Users/sergx/PycharmProjects/ITiSP0/weeks/week-17$

```

Рисунок 1.4 – Просмотр списка логов и сводки

Заключение

В результате работы был собран микросервисный проект для работы с логами. Реализованы внешний gateway, основной сервис logs-svc-s05 и внутренний сервис уведомлений log-notifier.

Проект использует REST для внешнего API и gRPC для внутреннего обмена между сервисами. Также была подготовлена контейнеризация через docker-compose и манифесты Kubernetes.

Итоговая система представляет собой цельный проект, готовый к запуску и проверке одной командой. В ней реализованы просмотр логов, сводка по уровням, фильтрация по уровню, удаление записи и получение истории уведомлений через /api/notifications. Это делает проект удобным для демонстрации взаимодействия REST и gRPC в одной системе.

Все тесты практических работ пройдены успешно, а созданная архитектура демонстрирует базовые принципы микросервисного подхода: разделение ответственности, отдельные интерфейсы взаимодействия и простую обработку недоступности внутреннего сервиса.